TP2: xv6 null-pointer dereference protection

Pedro Pires · Operating Systems, UFMG. Modified xv6-riscv (based on `mit-pdos/xv6-riscv`).

Stock xv6 links user programs at address `0x0`, so `*(int*)0` silently reads the program's first instruction. We make the first page of every address space invalid, so any access to it faults and kills the process.

1. xv6 modifications

Strategy: making the first page invalid

Page `[0x0, 0x1000)` is left **unmapped** (no valid PTE) and programs are moved one page up, to `0x1000`. An unmapped page costs no physical memory, and the MMU faults on any load, store or instruction fetch there. A constant names the new boundary:

```
kernel/memlayout.h
+// lowest valid user address; [0, USERBASE) is never mapped
+#define USERBASE PGSIZE
```

Kernel changes to relocate programs

`exec()` used to allocate memory from 0. It now starts at `USERBASE` and rejects ELF segments below it, which would otherwise map page 0 or panic in `loadseg()`:

```
kernel/exec.c
-  uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;
+  uint64 argc, sz = USERBASE, sp, ustack[MAXARG], stackbase;
   ...
+    if (ph.vaddr < USERBASE) // keep the null page unmapped
+      goto bad;
```

`fork()` (`uvmcopy`) and process teardown (`uvmfree`) already skip unmapped pages, because of lazy `sbrk`, so they needed no change. `/init` is started by `exec()`, so it is relocated too.

Build changes to relocate programs

```
user/user.ld
- . = 0x0;
+ . = 0x1000; /* leave page 0 unmapped (USERBASE in kernel/memlayout.h) */

Makefile
-       $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest ...
+       $(LD) $(LDFLAGS) -N -e main -Ttext 0x1000 -o $U/_forktest ...
+       $U/_nulltest\
```

All user programs use `user.ld`; `forktest` is linked by hand, so its `-Ttext` changes too. `readelf -l` confirms that every binary's first `LOAD` segment is now at `0x1000`.

Handling illegal references to the null address

Page 0 must also never be mapped *later*. Two paths could do it:

```
kernel/vm.c  (vmfault: lazy allocation on page fault)
-  if (va >= psz)
+  if (va < USERBASE || va >= psz) // never map the null page
     return 0;
kernel/proc.c  (growproc: sbrk with n < 0)
+    if (sz + n < USERBASE) // don't shrink into the null page
+      return -1;
```

Without the first check, the page-fault handler would treat a null dereference like a lazy `sbrk` page and map a fresh zeroed page at 0. Without the second, `sbrk(-sz)` followed by `sbrk(n)` would remap page 0. With both in place, the existing trap path does the rest:

- **User access:** a page fault (`scause` 12/13/15) reaches the `else` branch of `usertrap()`, which prints the fault and kills the process (exit status −1). Its memory is freed by the normal exit path.
- **System calls:** the kernel touches user memory only through `copyin` / `copyout` / `copyinstr`, which fail on page 0, so the call returns −1.

Overhead: no extra physical memory. The only runtime cost is one comparison on `exec`, `sbrk` shrink and page-fault paths.

2. Process address space

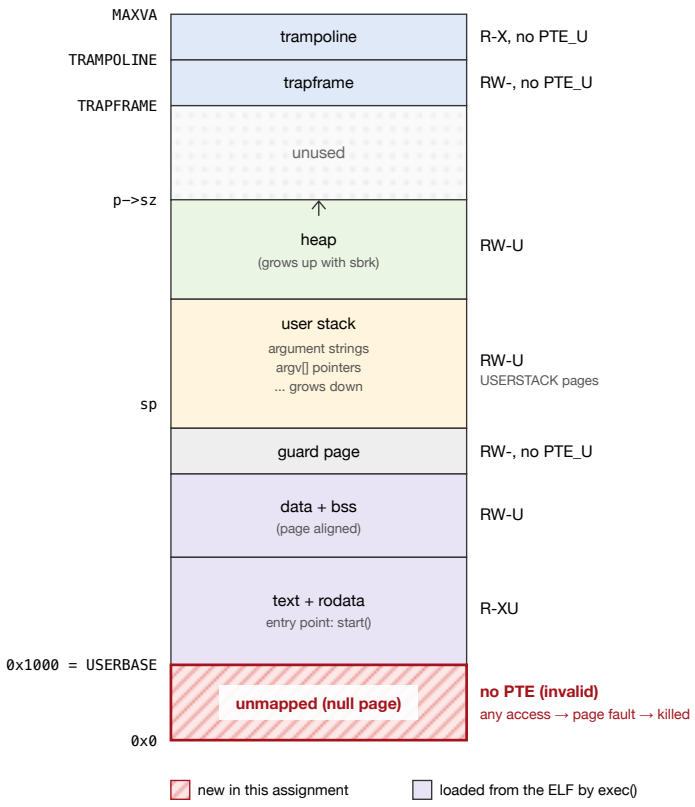


Figure 1. User address space with null-pointer protection (not to scale; `MAXVA` = 2^{38}). Only the bottom changes from Figure 3.4 of the xv6 book: page 0 is an unmapped hole and everything else is shifted up one page.

3. Tests

The new program `user/nulltest.c` runs each faulting case in a forked child and expects `wait()` to report status −1, the status the kernel sets when it kills a process. As a control, we also built it into an unmodified xv6:

Check	Modified	Original
Read of `0x0` and of `0xfff` kills the process	ok	FAILED (reads code, e.g. `0xe4061141`)
Write to / jump to `0x0` kills the process	ok	ok (by accident: read-only text, or the code crashes)
Program text starts at `0x1000`	ok	FAILED
`write` / `read` / `open` / `pipe` with a null pointer return −1	ok	FAILED (`write` copies code)
`sbrk` cannot shrink into page 0	ok	FAILED (test killed)
No leak: free pages equal before/after 30 killed children	ok	not reached

```
$ nulltest
usertrap(): unexpected scause 0xd pid=4
            sepc=0x103a stval=0x0
ok: read from address 0 kills the process
...
nulltest: ALL TESTS PASSED
```

Other checks. The full stock `usertests` suite (quick and slow) and `forktest` pass. `usertests` ends by comparing free pages before and after the whole run, which is a second leak check. Running an old binary still linked at `0x0` makes the shell print `exec oldcat failed` and the kernel keeps running.